Lab 6: The Timer Unit

10/15/2026

Daniel Xu
ECE 3218L

1 Overview

Timer1 is the ATmega328P's 16-bit timer. Its output compare unit can toggle a pin and request an interrupt when the counter TCNT1 matches OCR1A, and its input capture unit copies TCNT1 into ICR1 when an edge arrives on the ICP1 pin. In this lab, three programs were written using these features: a 440 Hz rectangular wave with a selectable duty cycle, a ten-note scale played on a piezo speaker, and a square wave at half the frequency of a function generator's signal.

2 Procedure

- **Pre-lab Questions** The handout's questions were answered before writing Program 1:
  (a) $f_{cc} = 16$ MHz.
  (b) 1, 8, 64, 256, or 1024, selected with CS12:10 in TCCR1B.
  (c) $f_{tc} = f_{cc}/sp$.
  (d) $pw = 1/f_w = 1/440$ Hz $= 2.273$ ms.
  (e) count $= f_{tc} \cdot pw$.
  (f) count $= f_{cc} \cdot pw/sp$.
  (g) Timer1 is 16 bits, so the max value is $2^{16} - 1 = 65535$.
  (h) With $sp = 1$, count $= 16$ MHz$/440$ Hz $= 36364 < 65536$, so $sp = 1$.
  (i) count $= (16$ MHz$) \cdot (1/440$ Hz$)/1 = 36363.6 \approx 36364$, which fits within our 16-bit register.
- **Program 1** The subroutine SETWAVE takes the period pw in clicks (r25:r24), a duty cycle code (r23), and a prescaler code (r22). The high width is pw shifted right 3, 2, or 1 times for 12.5%, 25%, or 50%, and the low width is pw minus the high width. For 75% and 87.5%, bit 7 of the code swaps the two widths.

  Timer1 runs in CTC mode with OC1A set to toggle on each compare match. Each part lasts OCR1A + 1 clicks in CTC mode, so SETWAVE stores each width minus one.

  On each compare match, the hardware toggles OC1A and clears TCNT1, and the ISR loads OCR1A with the width of the part that just started: LOWCNT (width of low) if PB1 is now low and HIGHCNT (width of high) if it is high.
- **Program 2** The ten tone periods in Table 2 are copied from program memory into SRAM at startup. For each tone, the program calls SETWAVE at 50% duty, waits 500 ms with waitnms from Lab 5, stops the timer by clearing TCCR1B, and waits another 500 ms.

- **Program 3** Timer1 runs freely in normal mode at clk/8, so each click is 0.5 μs. CTC mode cannot be used because input capture and output compare share TCNT1. On each rising edge, the capture ISR stores ICR1 minus the previous capture in PERIOD. On each compare match, OC1A toggles and the compare ISR adds PERIOD to OCR1A, so the output toggles once per input period and has half the input frequency.

3 Results

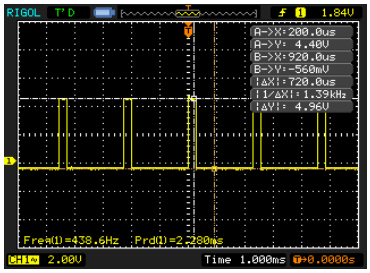
3.1 Program 1

Figure 1 shows the output at each duty cycle. The scope measured 438.6 Hz (2.280 ms) for all five. The time on (t_{on}) and time off (t_{off}) were measured from the traces, where one period spans 57 pixels at 1 ms/div, or 40 μs per pixel. The duty cycle is then

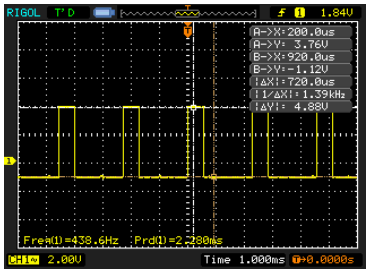
$$D = \frac{t_{\text{on}}}{t_{\text{on}} + t_{\text{off}}}. \tag{1}$$

Table 1: Program 1 at 440 Hz ($pw = 36364$ clicks, prescaler 1). Measured times are read from the traces in Figure 1, and measured voltages are the cursor readings.

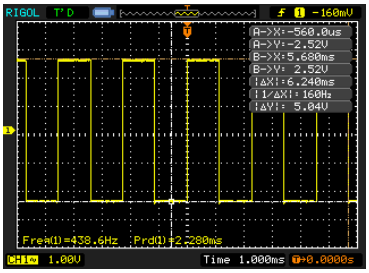
	Expected		Measured			
Duty cycle	t_{on}	t_{off}	t_{on}	t_{off}	Low / high (V)	D from Eq. 1
12.5%	284.1 μs	1.989 ms	280 μs	2.00 ms	−0.56 / 4.40	12.3%
25%	568.2 μs	1.705 ms	560 μs	1.72 ms	−1.12 / 3.76	24.6%
50%	1.136 ms	1.136 ms	1.14 ms	1.14 ms	−2.52 / 2.52	50.0%
75%	1.705 ms	568.2 μs	1.72 ms	560 μs	−3.68 / 1.28	75.4%
87.5%	1.989 ms	284.1 μs	2.00 ms	280 μs	−4.32 / 0.64	87.7%



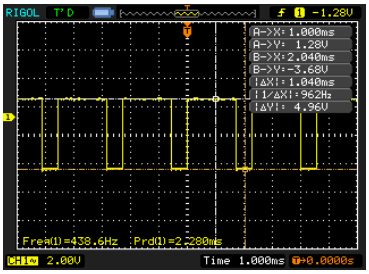
(a) 12.5% duty cycle



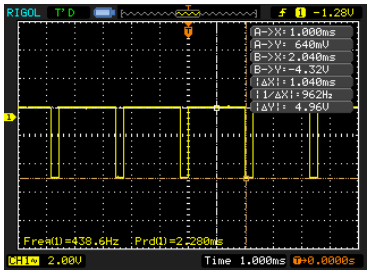
(b) 25% duty cycle



(c) 50% duty cycle



(d) 75% duty cycle



(e) 87.5% duty cycle

Figure 1: Program 1 output on OC1A. The scope measured 438.6 Hz (2.280 ms) at every duty cycle.

3.2 Program 2

The program played ten rising notes from C4 to E5, each lasting half a second with half a second of silence in between. The periods were stored in program memory then copied to SRAM based on Table 2.

Table 2: Tone periods at prescaler 1 ($pw = 16\ \text{MHz}/f$).

Note	Frequency (Hz)	pw (clicks)
C4	261.63	61155
D4	293.66	54485
E4	329.63	48539
F4	349.23	45815
G4	392.00	40816
A4	440.00	36364
B4	493.88	32397
C5	523.25	30578
D5	587.33	27242
E5	659.25	24270

3.3 Program 3

The generator was set to 1.012 kHz (Figure 2), and the output measured 504.6 Hz with a period of 1.982 ms (Figure 3). The expected output is $1012/2 = 506.0$ Hz, with a period of 1.976 ms.

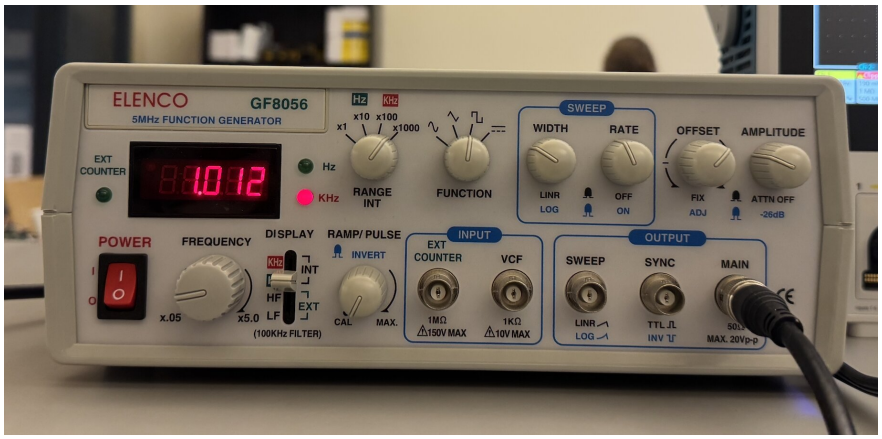


Figure 2: Function generator driving ICP1 at 1.012 kHz.

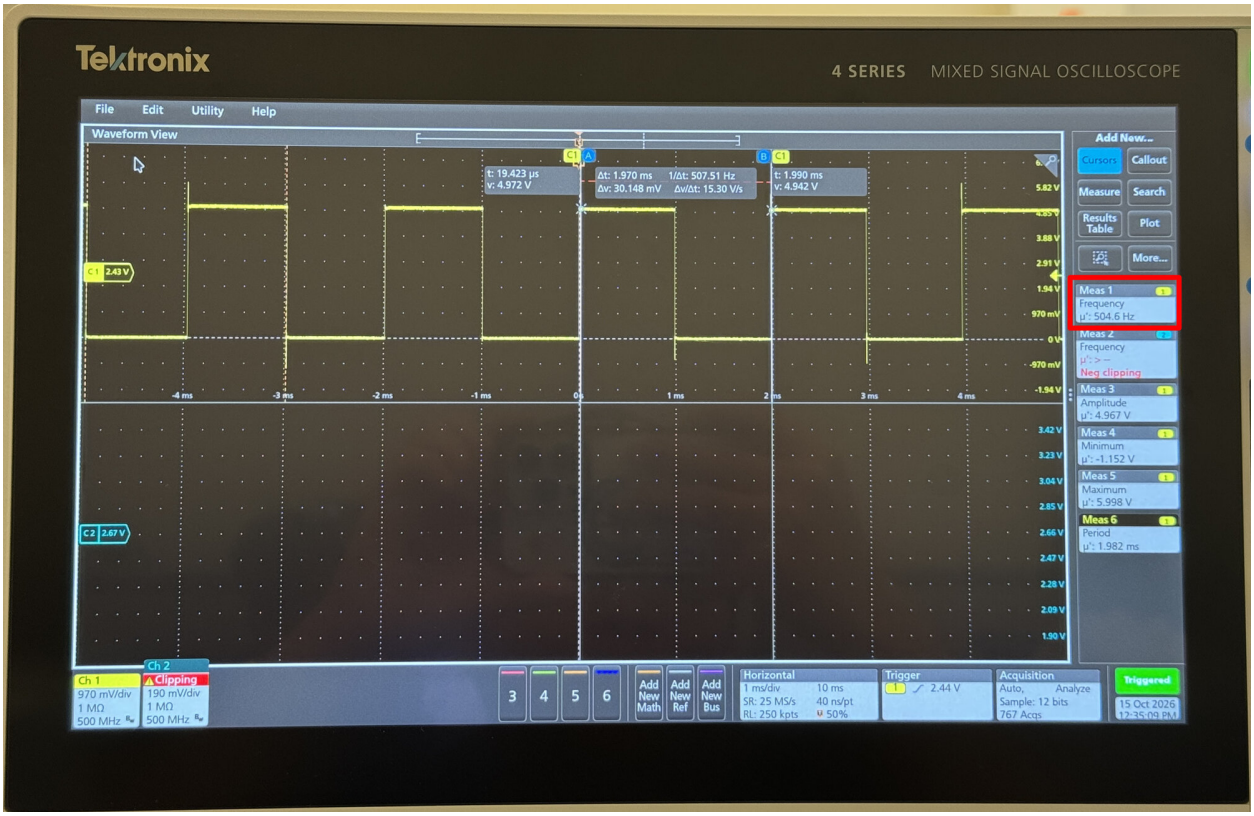


Figure 3: Program 3 output on OC1A. Frequency is 504.6 Hz (Meas 1, outlined in red), period is 1.982 ms (Meas 6), and amplitude is 4.967 V (Meas 3).

4 Post-Lab

Program 1: Are the results what you expected? Can you see any deviation from what you expected? Why (or why not)?
Yes. Each part of the wave is a whole number of timer clicks, and the hardware toggles OC1A at the compare match, so the ISR's response time does not affect the widths. The measured period of 2.280 ms is 0.32% longer than the expected 2.273 ms. This most likely comes from the board's 16 MHz clock running slightly slow, since every width scales with it. The clock may be running slow because of temperature swings or the built-in tolerance. The measured duty cycles are within 0.4 percentage points of their targets.

What are the limits of your program? What are the limits of the high and low portions of your signal? Why?

- **Longest period.** The period *pw* is stored in a 16-bit register, so the largest value is 65535 clicks. At $sp = 1$, that is 4.1 ms, or 244.1 Hz. Because SETWAVE treats *pw* as unsigned, this is double the 32767 limit in the handout.
- **Shortest high or low portion.** The ISR is the limit. After each toggle, the ISR has to load the next width into OCR1A before the timer counts past it, and it needs about 37 clock cycles to do that. If a portion is shorter than this, the timer passes the new value before it is loaded, misses the match, and keeps counting up to 65535 and around again, which adds a gap of up to 4.1 ms to the wave. At $sp = 1$, each portion should therefore be at least about

37 clicks (2.3 μs). For a 12.5% wave, the short portion is 1/8 of the period, so the period must be at least about $8 \times 37 \approx 300$ clicks, which limits the frequency to about 53 kHz.

- **Longest high or low portion.** The longest single portion is the long part of a 12.5% or 87.5% wave.

Program 2: What is the smallest and largest signal frequency that you can generate? Where are the limitations coming from?
The smallest frequency is set by the 16-bit compare register. At $sp = 1$, it is 16 MHz/65535 = 244.1 Hz. A prescaler of 1024 would lower this to 0.238 Hz.

The largest frequency is limited by the ISR. It runs twice per period, once for each toggle, and takes about 37 clock cycles each time. If each half of the wave is shorter than that (above about 216 kHz), the ISR runs nonstop and the main program never runs.

Program 3: Note the results and explain them.
The output was 504.6 Hz, 0.28% below the expected 506.0 Hz. Unlike Program 1, this result does not depend on the board's clock, because the input is measured and the output is generated with the same timer: the output always lasts exactly two input periods in clicks. This error is minimal and can be attributed to frequency drift in the generator or other factors.

What is the longest period that you can measure with the last program?
The capture ISR finds the period with a 16-bit subtraction (sub, then sbc). If the timer rolls over once between edges, the subtraction borrows, but because the arithmetic is modulo 2^{16}, the result is still the correct period. This works as long as the timer rolls over at most once per period, so the longest measurable period is

$$T_{max} = 65535 \times 0.5\ \mu\text{s} = 32.77\ \text{ms}, \tag{2}$$

which corresponds to an input of 30.5 Hz. With a prescaler of 1024, each click lasts 1024/16 MHz = 64 μs, so the longest measurable period would be

$$T_{max} = 65535 \times 64\ \mu\text{s} = 4.19\ \text{s}. \tag{3}$$

5 Discussion/Conclusion

This lab used Timer1's output compare and input capture units to generate and measure waveforms in hardware. All three programs worked as designed. Program 1 produced all five duty cycles at 438.6 Hz, which is only 0.32% below the target 440 Hz. Program 2 played the ten-note scale with the expected timing. Program 3 produced 504.6 Hz from a 1.012 kHz input and can measure periods of up to 32.77 ms.

A Full Code

Code 1: Program 1. Rectangular wave (main.asm).

```
.cseg
.org 0x0000
        jmp RESET           ; reset
.org OC1Aaddr
        jmp TIM1_COMPA      ; Timer1 compare match A

.org INT_VECTORS_SIZE       ; code starts after the vector table
.include "wave.inc"

.equ PW   = 36364           ; 440 Hz at prescaler 1
.equ DUTY = DC50            ; DC12, DC25, DC50, DC75 or DC87

RESET:
        ; point the stack at the top of SRAM
        ldi r16,high(RAMEND)
        out SPH,r16
        ldi r16,low(RAMEND)
        out SPL,r16

        ldi r24,low(PW)
        ldi r25,high(PW)
        ldi r23,DUTY
        ldi r22,CLK_1
        rcall SETWAVE

        ; the ISR does the rest
SPIN:
        rjmp SPIN
```

Code 2: SETWAVE subroutine and compare ISR, used by Programs 1 and 2 (wave.inc).

```
; wave.inc: rectangular wave on OC1A (PB1) using Timer1 output compare A
; must put ‘jmp TIM1_COMPA‘ at OC1Aaddr in its vector table.

; duty cycle codes for SETWAVE (r23)
; bits 1:0 = how far to shift pw right, bit 7 = invert the result
.equ DC12  = 0x03           ; 12.5% = pw >> 3
.equ DC25  = 0x02           ; 25%   = pw >> 2
.equ DC50  = 0x01           ; 50%   = pw >> 1
.equ DC75  = 0x82           ; 75%   = NOT 25%
.equ DC87  = 0x83           ; 87.5% = NOT 12.5%

; prescaler codes for SETWAVE (r22), the CS12:CS10 bits of TCCR1B
.equ CLK_1    = 1
.equ CLK_8    = 2
.equ CLK_64   = 3
.equ CLK_256  = 4
.equ CLK_1024 = 5

.dseg
HIGHCNT: .byte 2            ; OCR1A value for the high part (high width - 1)
LOWCNT:  .byte 2            ; OCR1A value for the low part (low width - 1)
.cseg

```

```
; SETWAVE: start a rectangular wave on OC1A
; r25:r24 = pw, period in timer clicks (at least 8)
; r23 = duty cycle code (DC12 ... DC87)
; r22 = prescaler code (CLK_1 ... CLK_1024)
; leaves r22-r25 unchanged and enables interrupts globally
SETWAVE:
        push r16
        push r17
        push r18
        push r19
        push r20
        push r21

        clr r16
        sts TCCR1B,r16      ; stop the timer while setting up

        ; part = pw >> shift
        movw r18,r24
        mov r20,r23
        andi r20,0x03       ; shift count
SW_SHIFT:
        lsr r19
        ror r18
        dec r20
        brne SW_SHIFT

        ; r19:r18 = high width = part, r17:r16 = low width = pw - part
        movw r16,r24
        sub r16,r18
        sbc r17,r19
        sbrs r23,7          ; inverted duty cycle?
        rjmp SW_SAVE
        movw r20,r16        ; yes -> swap high and low
        movw r16,r18
        movw r18,r20
SW_SAVE:
        ; in CTC mode each part lasts OCR1A + 1 clicks, so save width - 1
        subi r18,1
        sbci r19,0
        subi r16,1
        sbci r17,0
        sts HIGHCNT,r18
        sts HIGHCNT+1,r19
        sts LOWCNT,r16
        sts LOWCNT+1,r17

        ; PB1 is the output, starting high
        sbi DDRB,1
        sbi PORTB,1

        ; the first compare match ends the high part
        sts OCR1AH,r19      ; high byte first
        sts OCR1AL,r18
        clr r20
        sts TCNT1H,r20      ; count from 0
        sts TCNT1L,r20

        ; once COM1A is set, PB1 follows the OC1A bit, not PORTB,
        ; so force OC1A high: set-on-match mode + force compare,
```

```
        ; then switch to toggle mode (WGM11:10 = 0)
        ldi r20,(1<<COM1A1)|(1<<COM1A0)
        sts TCCR1A,r20
        ldi r20,(1<<FOC1A)
        sts TCCR1C,r20
        ldi r20,(1<<COM1A0) ; toggle OC1A on compare match
        sts TCCR1A,r20

        ldi r20,(1<<OCF1A)  ; clear a stale compare flag
        out TIFR1,r20
        ldi r20,(1<<OCIE1A) ; interrupt on compare match A
        sts TIMSK1,r20

        ; build TCCR1B in a register: CTC mode + prescaler
        ; writing it starts the timer, so do it last
        ldi r20,(1<<WGM12)
        or r20,r22
        sts TCCR1B,r20
        sei

        pop r21
        pop r20
        pop r19
        pop r18
        pop r17
        pop r16
        ret

; TIM1_COMPA: OC1A just toggled and TCNT1 was cleared
; load OCR1A with the width of the part that just started
TIM1_COMPA:
        push r16
        in r16,SREG
        push r16
        push r17

        sbis PINB,1         ; PB1 high -> low?
        rjmp TC_LOW
        lds r16,HIGHCNT     ; low -> high: time the high part
        lds r17,HIGHCNT+1
        rjmp TC_SET
TC_LOW:
        lds r16,LOWCNT      ; high -> low: time the low part
        lds r17,LOWCNT+1
TC_SET:
        sts OCR1AH,r17      ; high byte first
        sts OCR1AL,r16

        pop r17
        pop r16
        out SREG,r16
        pop r16
        reti
```

Code 3: Program 2. Tones (tones.asm).

```
; Program 2

```

```
.DEF ZERO=R0

.equ NTONES = 10

.dseg
TONES:  .byte 2*NTONES      ; tone periods in timer clicks, low byte first

.cseg
.org 0x0000
        jmp RESET           ; reset
.org OC1Aaddr
        jmp TIM1_COMPA      ; Timer1 compare match A

.org INT_VECTORS_SIZE       ; code starts after the vector table
.include "wave.inc"

RESET:
        ; point the stack at the top of SRAM
        ldi r16,high(RAMEND)
        out SPH,r16
        ldi r16,low(RAMEND)
        out SPL,r16
        clr ZERO            ; keep a constant 0 in r0

        ; copy the period table from flash into TONES
        ldi ZL,low(TONETAB*2)
        ldi ZH,high(TONETAB*2)
        ldi XL,low(TONES)
        ldi XH,high(TONES)
        ldi r16,2*NTONES
COPY:
        lpm r17,Z+
        st X+,r17
        dec r16
        brne COPY

        ; play each tone
        ldi YL,low(TONES)
        ldi YH,high(TONES)
        ldi r20,NTONES      ; tones left
NEXTTONE:
        ld r24,Y+           ; pw
        ld r25,Y+
        ldi r23,DC50
        ldi r22,CLK_1
        rcall SETWAVE       ; tone on

        ldi r24,low(500)
        ldi r25,high(500)
        rcall waitnms

        sts TCCR1B,ZERO     ; clear the prescaler bits -> timer stops, tone off

        ldi r24,low(500)
        ldi r25,high(500)
        rcall waitnms

        dec r20
        brne NEXTTONE
```

```
62
63  DONE:
64          rjmp DONE
65
66  ; tone periods at prescaler 1: 16 MHz / f
67  TONETAB:
68          .dw 61155           ; C4 261.63 Hz
69          .dw 54485           ; D4 293.66 Hz
70          .dw 48539           ; E4 329.63 Hz
71          .dw 45815           ; F4 349.23 Hz
72          .dw 40816           ; G4 392.00 Hz
73          .dw 36364           ; A4 440.00 Hz
74          .dw 32397           ; B4 493.88 Hz
75          .dw 30578           ; C5 523.25 Hz
76          .dw 27242           ; D5 587.33 Hz
77          .dw 24270           ; E5 659.25 Hz
78
79  ; 1 msec timer subroutine (exact; time includes call and return)
80  ; wait1ms expends 16000 clock cycles.
81  ; rcall wait1ms uses 3 clock cycles
82  ; Therefore from the rcall in the calling program through the
83  ; ret in the subroutine wait1ms expends
84  ; 3+1+1+1+(3996*4)[in loop]+3[end of loop]+1+1+1+4 = 16000cc
85  ; which at 16MHz (cc/s) equals 1ms.
86  wait1ms:
87          ldi zh,0x0F         ; 1cc
88          ldi zl,0x9D         ; 1cc
89          clz                 ; 1cc
90  waitloop:                   ; countdown 0x0F9D = 3997dec
91          sbiw z,1            ; 2cc
92          brne waitloop       ; 2cc true, 1cc false
93          nop                 ; 1cc
94          nop                 ; 1cc
95          nop                 ; 1cc
96          ret                 ; 4cc
97
98  ; waitnms: wait n ms
99  ; load n into r24, r25
100 ; r24, r25 and Z are restored before returning
101 ; the compare ISR runs during the wait and stretches it slightly
102 waitnms:
103         push r24            ; 2
104         push r25            ; 2
105         push zl             ; 2
106         push zh             ; 2
107         cp r24,ZERO         ; 1
108         cpc r25,ZERO        ; 1
109         breq waitnmsdone    ; 1 if n != 0
110 waitnmsloop:
111         rcall wait1ms       ; 16000 total with rcall and ret
112         sbiw r25:r24,1      ; 2
113         brne waitnmsloop    ; 2 if taken, 1 if not
114 waitnmsdone:
115         pop zh              ; 2
116         pop zl              ; 2
117         pop r25             ; 2
118         pop r24             ; 2
119         ret                 ; 4
```

Code 4: Program 3. Input capture (capture.asm).

```
; Program 3

.dseg
PREVCAP: .byte 2            ; ICR1 at the previous edge
PERIOD:  .byte 2            ; input period in clicks = output half period
ROLLED:  .byte 1            ; 1 if the timer rolled over during the last period

.cseg
.org 0x0000
        jmp RESET           ; reset
.org ICP1addr
        jmp TIM1_CAPT       ; Timer1 input capture
.org OC1Aaddr
        jmp TIM1_COMPA      ; Timer1 compare match A

.org INT_VECTORS_SIZE       ; code starts after the vector table
RESET:
        ; point the stack at the top of SRAM
        ldi r16,high(RAMEND)
        out SPH,r16
        ldi r16,low(RAMEND)
        out SPL,r16

        ; clear the variables
        clr r16
        sts PREVCAP,r16
        sts PREVCAP+1,r16
        sts PERIOD,r16
        sts PERIOD+1,r16
        sts ROLLED,r16

        cbi DDRB,0          ; PB0 = ICP1 input
        sbi DDRB,1          ; PB1 = OC1A output

        ldi r16,(1<<COM1A0) ; toggle OC1A on compare match, normal mode
        sts TCCR1A,r16

        ldi r16,(1<<ICF1)|(1<<OCF1A)    ; clear stale flags
        out TIFR1,r16
        ldi r16,(1<<ICIE1)|(1<<OCIE1A)  ; capture and compare A interrupts
        sts TIMSK1,r16

        ; noise canceller, capture on rising edge, clk/8 (starts the timer)
        ldi r16,(1<<ICNC1)|(1<<ICES1)|(1<<CS11)
        sts TCCR1B,r16
        sei

        ; the ISRs do the rest
SPIN:
        rjmp SPIN

; TIM1_CAPT: a rising edge arrived on ICP1
; PERIOD = ICR1 - previous ICR1
TIM1_CAPT:
        push r16
        in r16,SREG
        push r16
```

```
58          push r17
59          push r18
60          push r19
61
62          lds r16,ICR1L       ; low byte first
63          lds r17,ICR1H
64          lds r18,PREVCAP
65          lds r19,PREVCAP+1
66          sts PREVCAP,r16     ; this edge is the next one's previous edge
67          sts PREVCAP+1,r17
68
69          ; now - prev; borrow (C = 1) means now < prev, so the timer rolled
70          ; over once. The 16-bit result already equals now + 65536 - prev,
71          ; which is the right period, so only the flag is recorded.
72          sub r16,r18
73          sbc r17,r19
74          clr r18             ; clr leaves C alone
75          rol r18             ; r18 = rollover flag
76          sts ROLLED,r18
77
78          sts PERIOD,r16
79          sts PERIOD+1,r17
80
81          pop r19
82          pop r18
83          pop r17
84          pop r16
85          out SREG,r16
86          pop r16
87          reti
88
89  ; TIM1_COMPA: OC1A just toggled; schedule the next toggle
90  ; OCR1A = OCR1A + PERIOD. Adding to OCR1A instead of TCNT1 keeps the
91  ; ISR's own latency from adding to every half period.
92  TIM1_COMPA:
93          push r16
94          in r16,SREG
95          push r16
96          push r17
97          push r18
98          push r19
99
100         lds r16,OCR1AL
101         lds r17,OCR1AH
102         lds r18,PERIOD
103         lds r19,PERIOD+1
104         add r16,r18
105         adc r17,r19
106         sts OCR1AH,r17      ; high byte first
107         sts OCR1AL,r16
108
109         pop r19
110         pop r18
111         pop r17
112         pop r16
113         out SREG,r16
114         pop r16
115         reti
```